

Universidad Politécnica de San Luis Potosí

LAB: FILE PATH TRAVERSAL SIMPLE CASE

Presentado por:

Christian Alfredo Morales Meza - 184937

Asignatura: Fundamentos de Ciberseguridad

Profesor: Servando López Contreras

Fecha de entrega: 09/09/2026

Índice de contenido

Índice de contenido 2

Ficha del laboratorio 3

Objetivo del laboratorio 3

Explicación técnica de la vulnerabilidad 3

Procedimiento paso a paso..... 3

 Evidencia del procedimiento..... 4

Evidencias obligatorias 9

Explicación del payload..... 11

Mitigación recomendada..... 12

Ficha del laboratorio

Laboratorio: LAB: FILE PATH TRAVERSAL SIMPLE CASE

Nivel: Apprentice

Tipo de explotación: File Path Traversal (Directory Traversal)

Objetivo del laboratorio

Recuperar y visualizar el contenido del archivo sensible `/etc/passwd` del sistema operativo mediante la manipulación del parámetro `filename` en la funcionalidad de carga de imágenes.

Explicación técnica de la vulnerabilidad

La vulnerabilidad de salto de directorio (Path Traversal o Directory Traversal) se origina en una validación deficiente o nula de las entradas provistas por el usuario antes de ser procesadas directamente por las interfaces de programación de aplicaciones (APIs) del sistema de archivos del servidor. En una implementación segura, las peticiones de recursos estáticos deben resolverse dentro de un directorio base confinado (sandboxed). Sin embargo, cuando el backend concatena de forma directa el parámetro de entrada con una ruta estática en el almacenamiento local sin sanear secuencias de control de rutas, el sistema operativo interpreta secuencias como `'../'` como instrucciones de navegación relativa para retroceder un nivel hacia el directorio padre en el árbol de carpetas.

En el marco de OWASP Top 10:2021, esta debilidad se clasifica formalmente bajo la categoría A01:2021 - Broken Access Control (Control de Acceso Roto). Al evaluar la triada CIA, el impacto principal e inmediato recae sobre la Confidencialidad, dado que permite la exfiltración arbitraria de archivos del sistema operativo, configuraciones, credenciales y código fuente. No obstante, si la aplicación permite además operaciones de escritura sobre rutas manipuladas, la Integridad y la Disponibilidad se ven críticamente comprometidas.

El riesgo real en un entorno corporativo o productivo es severo: un atacante puede extraer archivos como `/etc/passwd` o `/etc/shadow` para obtener listas de usuarios y hashes de contraseñas, descubrir secretos de infraestructura en archivos `.env` o ficheros de configuración de bases de datos, y en escenarios avanzados donde concurre la subida de archivos o la sobreescritura de binarios/crons, evolucionar hacia una Ejecución Remota de Código (RCE) que resulte en la toma de control total del servidor.

Procedimiento paso a paso

Se inició el entorno de auditoría abriendo Burp Suite Community Edition y asegurando la configuración adecuada del proxy integrado para registrar el tráfico generado entre el navegador y la aplicación web vulnerable.

Se navegó a la página principal de la tienda en línea provista por el laboratorio, la cual despliega un catálogo de productos con sus respectivas imágenes ilustrativas, verificando que el reto se encontraba en estado 'Not solved'.

Al monitorear las peticiones generadas automáticamente por el navegador durante el renderizado de la página, se identificaron múltiples solicitudes HTTP concurrentes dirigidas al endpoint '/image', las cuales pasaban el nombre del recurso gráfico a través del parámetro 'filename' (por ejemplo, filename=7.jpg).

Se analizó el historial HTTP en la pestaña Proxy de Burp Suite, seleccionando una de las peticiones representativas que interactuaban con dicho endpoint para evaluar la mecánica de recuperación de archivos en el backend.

Mediante el menú contextual, se envió la petición seleccionada al módulo Repeater de Burp Suite con el atajo de teclado Ctrl+R, con el fin de iniciar las pruebas de manipulación de parámetros de forma aislada y controlada.

En Repeater, se ejecutó una primera prueba enviando una ruta absoluta directa (/etc/passwd) en el parámetro 'filename'. El servidor web respondió con un código 'HTTP/2 400 Bad Request' y el mensaje JSON 'No such file', lo que confirmó que el backend añade la entrada a un directorio base predefinido (tal como /var/www/images/) en lugar de ejecutar la consulta desde la raíz absoluta del sistema.

Identificado el comportamiento relativo, se procedió a inyectar una secuencia de escape de directorios conformada por retrocesos relativos sucesivos ('../../etc/passwd') para salir del directorio de imágenes y alcanzar la raíz del sistema de archivos.

Al emitir la solicitud con la carga útil estructurada, el servidor respondió con un código 'HTTP/2 200 OK' y el encabezado Content-Type: image/jpeg, retornando en el cuerpo del mensaje el contenido en texto plano del archivo '/etc/passwd' del sistema operativo Linux.

Finalmente, tras completarse con éxito la lectura no autorizada del archivo del sistema, la plataforma de PortSwigger detectó la exfiltración y actualizó el estado del laboratorio a 'Solved'.

Evidencia del procedimiento

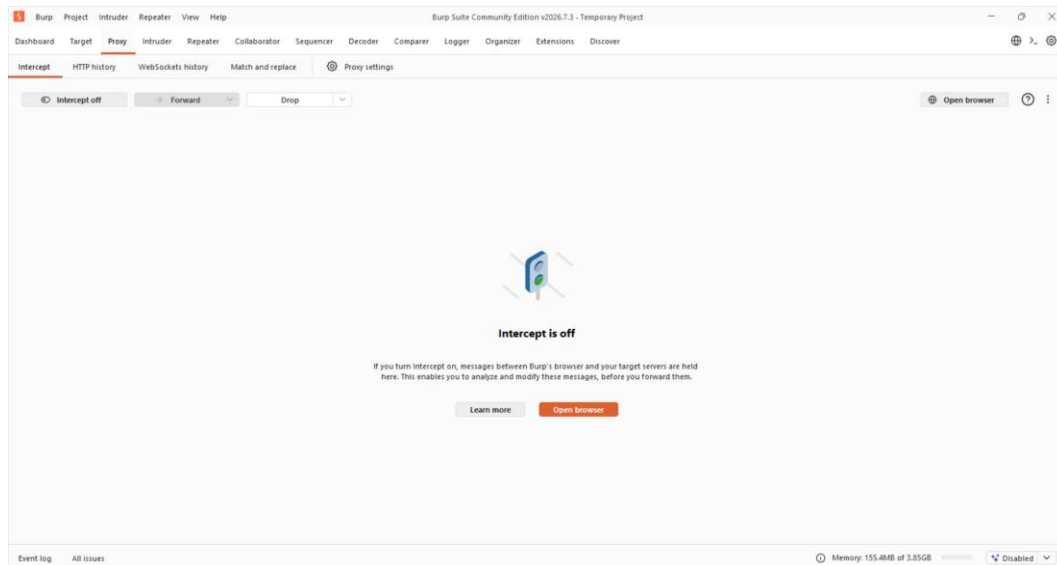


Figura 1. Configuración inicial del módulo Proxy en Burp Suite con la interceptación desactivada para monitorear el tráfico pasivo.

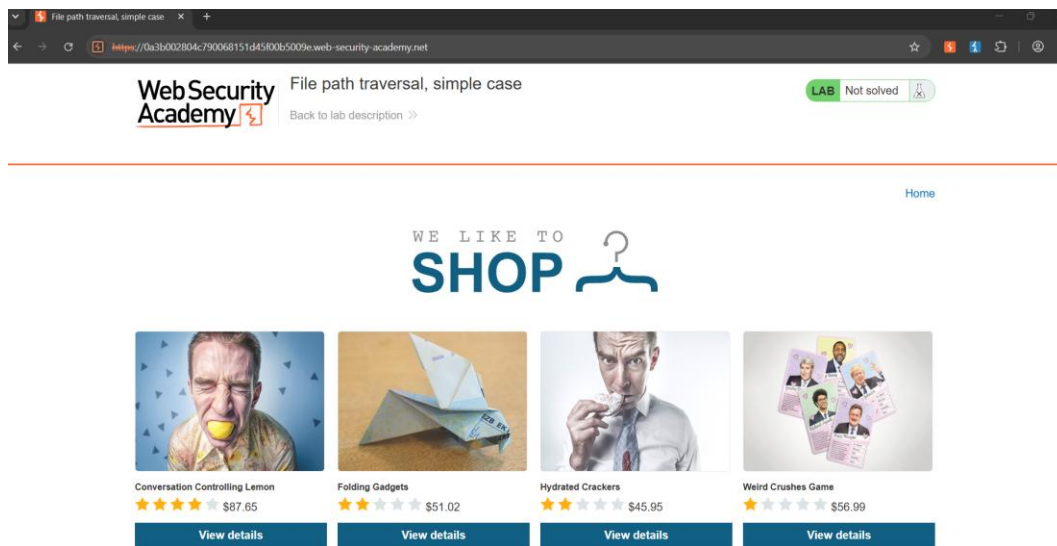


Figura 2. Acceso al portal de la tienda en línea en el navegador embebido para inspeccionar la carga inicial de los productos.

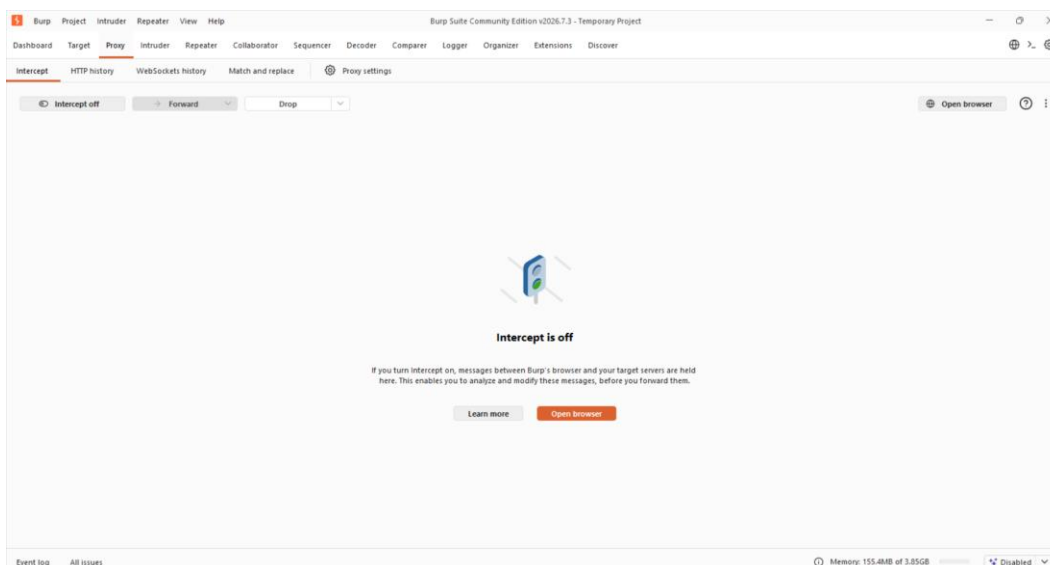


Figura 3. Verificación del estado de escucha del Proxy de Burp Suite antes de capturar las transacciones de los recursos estáticos.

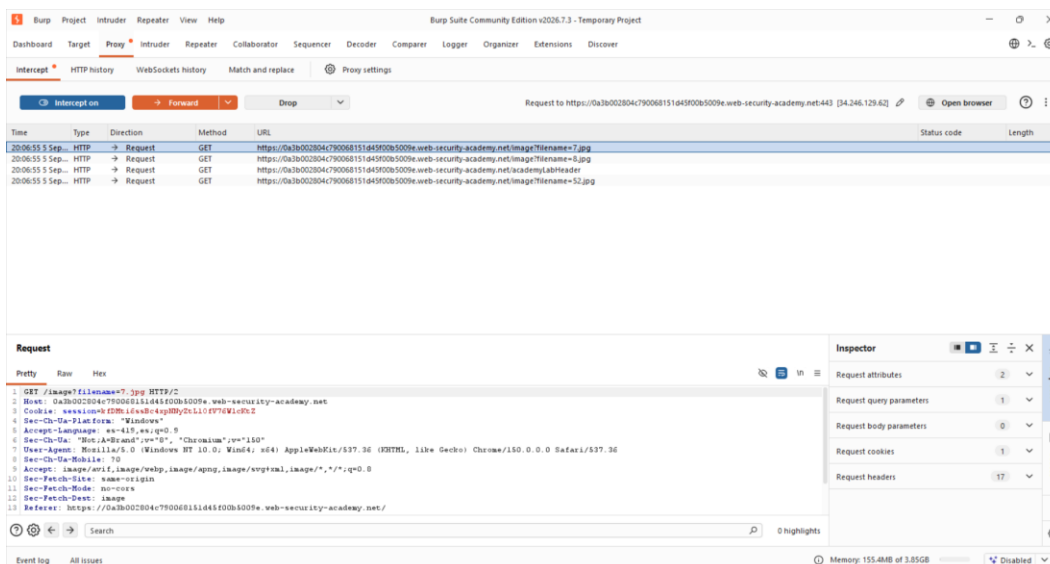


Figura 4. Interceptación de la petición GET dirigida al endpoint /image con el parámetro filename apuntando a una imagen estándar.

The screenshot shows the 'HTTP history' tab in Burp Suite. The table lists 33 requests, all of which are GET requests to the same host (https://0a3b002004c7900601...). The requests are filtered to show only those with the extension '.jpg' in the URL. The 'Status code' column shows that all requests were successful (200). The 'Length' column shows the size of the response in bytes. The 'MIME type' column shows the content type of the response. The 'Extension' column shows the file extension of the response. The 'Title' column shows the title of the response. The 'Notes' column shows the notes for each request. The 'TLS' column shows the TLS version used. The 'IP' column shows the IP address of the server. The 'Cookies' column shows the cookies sent with the request. The 'Time' column shows the time taken for the request to complete. The 'Listener port' column shows the port used by the listener. The 'Start response' column shows the start of the response.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response
1	https://0a3b002004c7900601...	GET	/			200	1004	HTML	css	File path traversal, sim...		✓	34.246.129.62		20:06:47 S...	8000	206
2	https://0a3b002004c7900601...	GET	/resources/fabheader/css/academyl...			200	5771	CS	css			✓	34.246.129.62		20:06:55 S...	8000	199
3	https://0a3b002004c7900601...	GET	/resources/fabheader/js/fabheader.js			200	1673	script	js			✓	34.246.129.62		20:06:55 S...	8000	249
4	https://0a3b002004c7900601...	GET	/resources/css/fabCommerce.css			200	28347	CS	css			✓	34.246.129.62		20:06:55 S...	8000	200
5	https://0a3b002004c7900601...	GET	/resources/images/shop.svg			200	7250	SVG	svg			✓	34.246.129.62		20:06:55 S...	8000	199
6	https://0a3b002004c7900601...	GET	/image?filename=7.jpg		✓	200	194100	JPEG				✓	34.246.129.62		20:06:55 S...	8000	1067
7	https://0a3b002004c7900601...	GET	/resources/images/rating4.png			200	742	PNG	png			✓	34.246.129.62		20:06:55 S...	8000	200
8	https://0a3b002004c7900601...	GET	/image?filename=8.jpg		✓	200	166301	JPEG				✓	34.246.129.62		20:06:55 S...	8000	240
9	https://0a3b002004c7900601...	GET	/resources/images/rating3.png			200	740	PNG	png			✓	34.246.129.62		20:06:55 S...	8000	199
10	https://0a3b002004c7900601...	GET	/academylabheader		✓	400	130	JSON				✓	34.246.129.62		20:06:55 S...	8000	208
11	https://0a3b002004c7900601...	GET	/image?filename=52.jpg		✓	200	143243	JPEG				✓	34.246.129.62		20:06:55 S...	8000	1257
12	https://0a3b002004c7900601...	GET	/image?filename=62.jpg		✓	200	174629	JPEG				✓	34.246.129.62		20:07:24 S...	8000	226
13	https://0a3b002004c7900601...	GET	/resources/fabheader/images/logo...			200	8852	SVG	svg			✓	34.246.129.62		20:07:24 S...	8000	255
14	https://0a3b002004c7900601...	GET	/resources/fabheader/images/logo...			200	942	SVG	svg			✓	34.246.129.62		20:07:24 S...	8000	255
15	https://0a3b002004c7900601...	GET	/resources/fabheader/images/logo...			200	708	PNG	png			✓	34.246.129.62		20:07:24 S...	8000	308
16	https://0a3b002004c7900601...	GET	/image?filename=15.jpg		✓	200	313445	JPEG				✓	34.246.129.62		20:07:24 S...	8000	803
17	https://0a3b002004c7900601...	GET	/resources/images/rating3.png			200	758	PNG	png			✓	34.246.129.62		20:07:24 S...	8000	371
18	https://0a3b002004c7900601...	GET	/image?filename=31.jpg		✓	200	127647	JPEG				✓	34.246.129.62		20:07:25 S...	8000	997
19	https://0a3b002004c7900601...	GET	/image?filename=36.jpg		✓	200	152676	JPEG				✓	34.246.129.62		20:07:40 S...	8000	643
20	https://0a3b002004c7900601...	GET	/image?filename=23.jpg		✓	200	266000	JPEG				✓	34.246.129.62		20:07:40 S...	8000	841
21	https://0a3b002004c7900601...	GET	/resources/images/rating5.png			200	624	PNG	png			✓	34.246.129.62		20:07:40 S...	8000	997
22	https://0a3b002004c7900601...	GET	/image?filename=32.jpg		✓	200	87834	JPEG				✓	34.246.129.62		20:07:41 S...	8000	231
23	https://0a3b002004c7900601...	GET	/image?filename=10.jpg		✓	200	146550	JPEG				✓	34.246.129.62		20:07:44 S...	8000	219
24	https://0a3b002004c7900601...	GET	/image?filename=21.jpg		✓	200	194766	JPEG				✓	34.246.129.62		20:07:44 S...	8000	247
25	https://0a3b002004c7900601...	GET	/image?filename=53.jpg		✓	200	163688	JPEG				✓	34.246.129.62		20:07:46 S...	8000	219
26	https://0a3b002004c7900601...	GET	/image?filename=29.jpg		✓	200	123333	JPEG				✓	34.246.129.62		20:07:46 S...	8000	214
27	https://0a3b002004c7900601...	GET	/image?filename=39.jpg		✓	200	150621	JPEG				✓	34.246.129.62		20:07:46 S...	8000	236
28	https://0a3b002004c7900601...	GET	/image?filename=54.jpg		✓	200	163137	JPEG				✓	34.246.129.62		20:07:46 S...	8000	235
29	https://0a3b002004c7900601...	GET	/image?filename=51.jpg		✓	200	301401	JPEG				✓	34.246.129.62		20:07:47 S...	8000	229
30	https://0a3b002004c7900601...	GET	/image?filename=8.jpg		✓	200	110082	JPEG				✓	34.246.129.62		20:07:47 S...	8000	230
31	https://0a3b002004c7900601...	GET	/image?filename=43.jpg		✓	200	412390	JPEG				✓	34.246.129.62		20:07:47 S...	8000	233
32	https://0a3b002004c7900601...	GET	/image?filename=63.jpg		✓	200	147724	JPEG				✓	34.246.129.62		20:07:48 S...	8000	218
33	https://0a3b002004c7900601...	GET	/image?filename=22.jpg		✓	200	104930	JPEG				✓	34.246.129.62		20:07:48 S...	8000	212

Figura 5. Inspección del historial HTTP en el Proxy para identificar las llamadas recurrentes hacia el recurso vulnerable /image.

The screenshot shows the 'HTTP history' tab in Burp Suite. The table lists 33 requests, all of which are GET requests to the same host (https://0a3b002004c7900601...). The requests are filtered to show only those with the extension '.jpg' in the URL. The 'Status code' column shows that all requests were successful (200). The 'Length' column shows the size of the response in bytes. The 'MIME type' column shows the content type of the response. The 'Extension' column shows the file extension of the response. The 'Title' column shows the title of the response. The 'Notes' column shows the notes for each request. The 'TLS' column shows the TLS version used. The 'IP' column shows the IP address of the server. The 'Cookies' column shows the cookies sent with the request. The 'Time' column shows the time taken for the request to complete. The 'Listener port' column shows the port used by the listener. The 'Start response' column shows the start of the response.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response
1	https://0a3b002004c7900601...	GET	/			200	1004	HTML	css	File path traversal, sim...		✓	34.246.129.62		20:06:47 S...	8000	206
2	https://0a3b002004c7900601...	GET	/resources/fabheader/css/academyl...			200	5771	CS	css			✓	34.246.129.62		20:06:55 S...	8000	199
3	https://0a3b002004c7900601...	GET	/resources/fabheader/js/fabheader.js			200	1673	script	js			✓	34.246.129.62		20:06:55 S...	8000	249
4	https://0a3b002004c7900601...	GET	/resources/css/fabCommerce.css			200	28347	CS	css			✓	34.246.129.62		20:06:55 S...	8000	200
5	https://0a3b002004c7900601...	GET	/resources/images/shop.svg			200	7250	SVG	svg			✓	34.246.129.62		20:06:55 S...	8000	199
6	https://0a3b002004c7900601...	GET	/image?filename=7.jpg		✓	200	194100	JPEG				✓	34.246.129.62		20:06:55 S...	8000	1067
7	https://0a3b002004c7900601...	GET	/resources/images/rating4.png			200	742	PNG	png			✓	34.246.129.62		20:06:55 S...	8000	200
8	https://0a3b002004c7900601...	GET	/image?filename=8.jpg		✓	200	166301	JPEG				✓	34.246.129.62		20:06:55 S...	8000	240
9	https://0a3b002004c7900601...	GET	/resources/images/rating3.png			200	740	PNG	png			✓	34.246.129.62		20:06:55 S...	8000	199
10	https://0a3b002004c7900601...	GET	/academylabheader		✓	400	130	JSON				✓	34.246.129.62		20:06:55 S...	8000	208
11	https://0a3b002004c7900601...	GET	/image?filename=52.jpg		✓	200	143243	JPEG				✓	34.246.129.62		20:06:55 S...	8000	1257
12	https://0a3b002004c7900601...	GET	/image?filename=62.jpg		✓	200	174629	JPEG				✓	34.246.129.62		20:07:24 S...	8000	226
13	https://0a3b002004c7900601...	GET	/resources/fabheader/images/logo...			200	8852	SVG	svg			✓	34.246.129.62		20:07:24 S...	8000	255
14	https://0a3b002004c7900601...	GET	/resources/fabheader/images/logo...			200	942	SVG	svg			✓	34.246.129.62		20:07:24 S...	8000	255
15	https://0a3b002004c7900601...	GET	/resources/fabheader/images/logo...			200	708	PNG	png			✓	34.246.129.62		20:07:24 S...	8000	308
16	https://0a3b002004c7900601...	GET	/image?filename=15.jpg		✓	200	313445	JPEG				✓	34.246.129.62		20:07:24 S...	8000	803
17	https://0a3b002004c7900601...	GET	/resources/images/rating3.png			200	758	PNG	png			✓	34.246.129.62		20:07:24 S...	8000	371
18	https://0a3b002004c7900601...	GET	/image?filename=31.jpg		✓	200	127647	JPEG				✓	34.246.129.62		20:07:25 S...	8000	997
19	https://0a3b002004c7900601...	GET	/image?filename=36.jpg		✓	200	152676	JPEG				✓	34.246.129.62		20:07:40 S...	8000	643
20	https://0a3b002004c7900601...	GET	/image?filename=23.jpg		✓	200	266000	JPEG				✓	34.246.129.62		20:07:40 S...	8000	841

The 'Request' tab is selected, showing the details of the selected request (18). The 'Raw' tab is also visible, showing the raw HTTP request. The 'Inspector' tab is also visible, showing the request attributes, query parameters, cookies, headers, and response headers.

Figura 6. Envío de la petición representativa GET /image?filename=31.jpg al módulo Repeater para manipular sus parámetros.

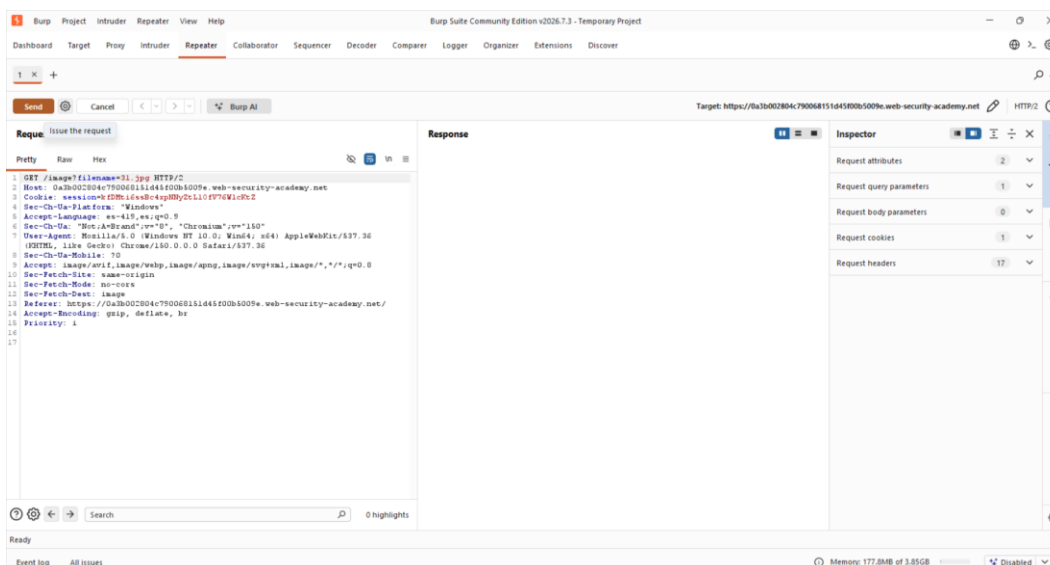


Figura 7. Recepción y visualización del request original en Repeater, listo para la fase de pruebas de inyección.

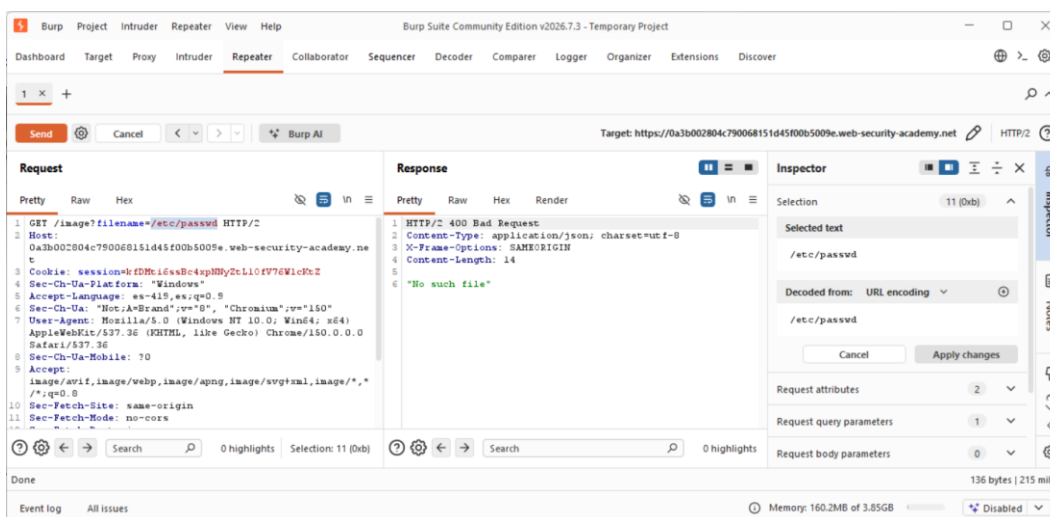


Figura 8. Envío de una ruta absoluta (/etc/passwd) como sondeo inicial, recibiendo un error 400 Bad Request que evidencia una concatenación relativa.

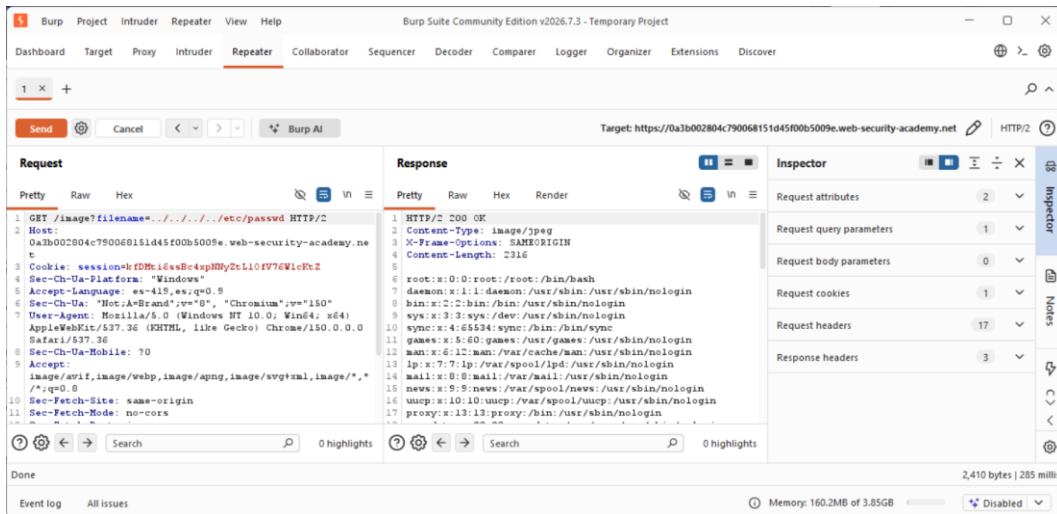


Figura 9. Ejecución exitosa del payload de path traversal en Repeater, obteniendo el archivo `/etc/passwd` con respuesta HTTP 200 OK.

Evidencias obligatorias

Request interceptado

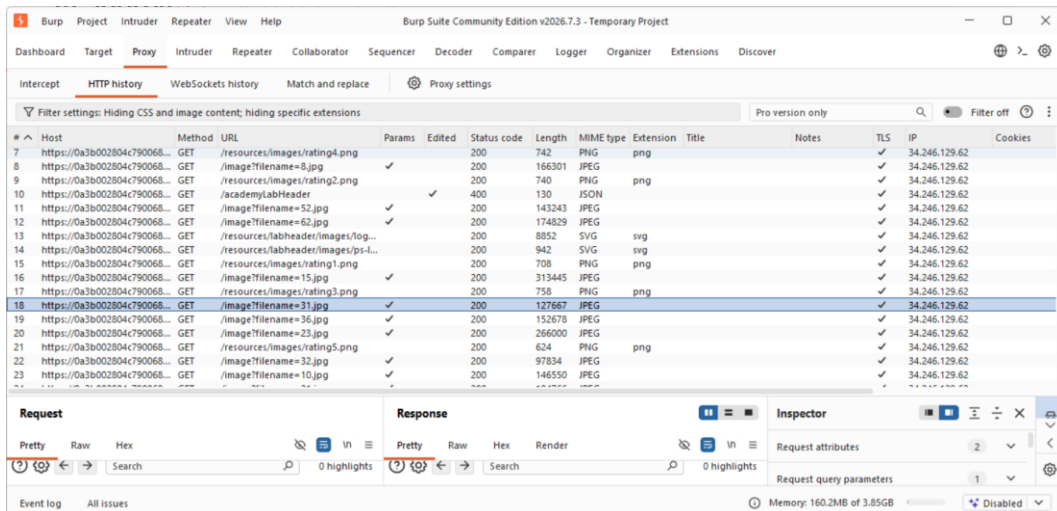


Figura 10. Request interceptado — Petición HTTP GET legítima capturada en Burp Suite dirigida a `/image` solicitando una imagen mediante el parámetro `filename=31.jpg`.

Payload utilizado

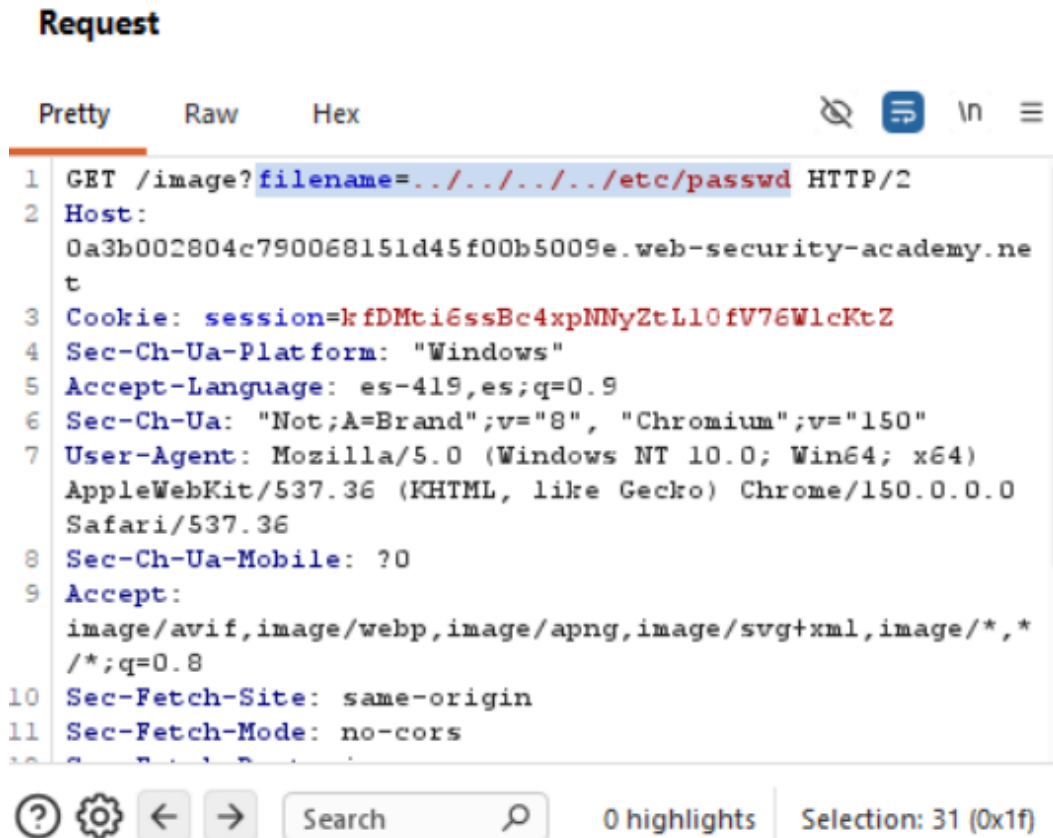


Figura 11. Payload utilizado — Carga útil inyectada en el parámetro filename consistente en secuencias de salto de directorio relativas hacia /etc/passwd.

Respuesta vulnerable

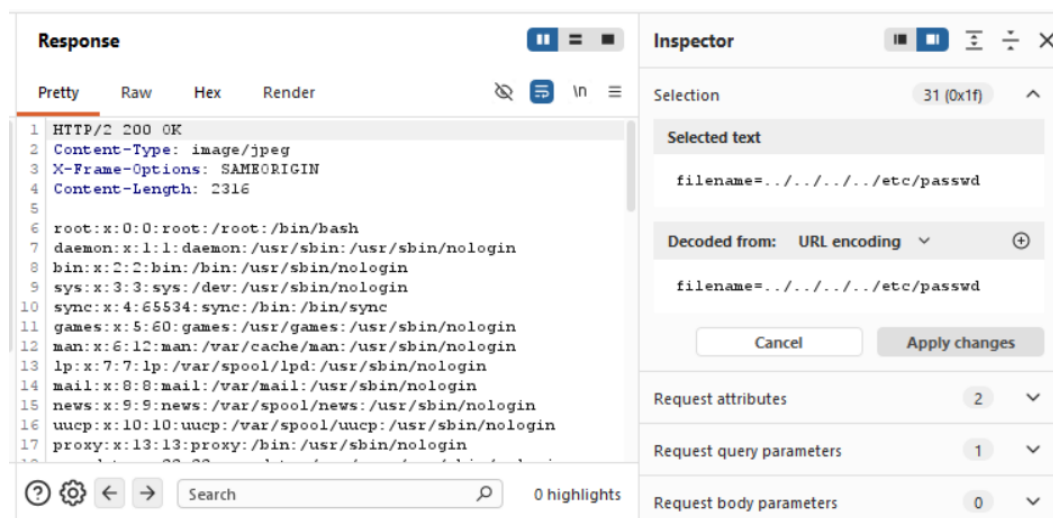


Figura 12. Respuesta vulnerable — Respuesta HTTP/2 200 OK del servidor web que expone en su cuerpo la lista de usuarios y configuraciones del archivo /etc/passwd.

Confirmación de “Lab Solved”

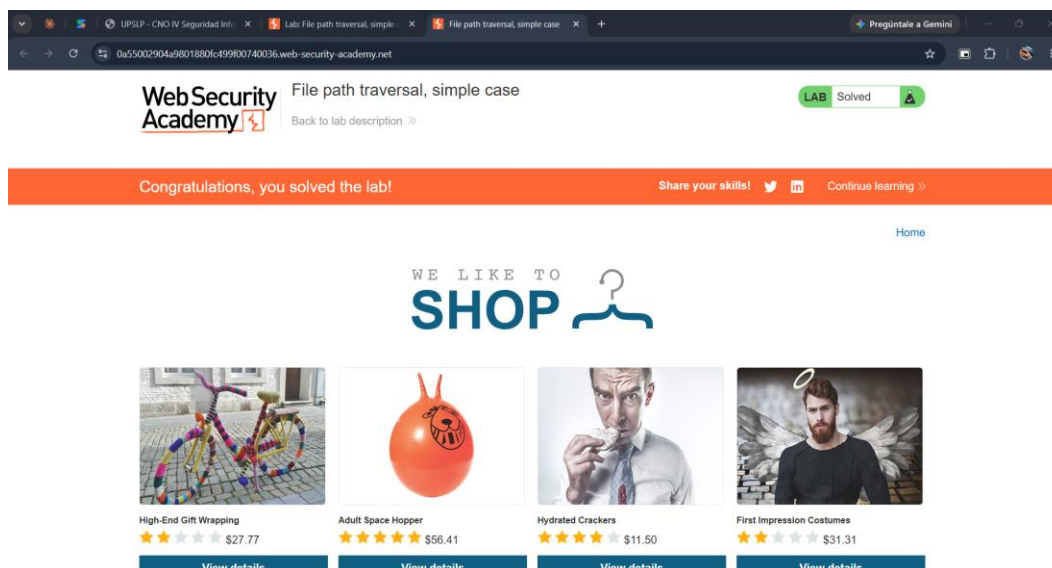


Figura 13. Confirmación de “Lab Solved” — Notificación oficial de PortSwigger Web Security Academy en la cabecera del laboratorio confirmando la resolución exitosa del ejercicio.

Explicación del payload

El vector de ataque utilizado es: `'../../etc/passwd'` (o su variante extendida `'../../etc/passwd')`.

Análisis sintáctico y funcional:

1. `'../'`: Representa la directiva estándar POSIX y de sistemas tipo UNIX para referenciar al directorio padre del directorio de trabajo actual. Cada repetición de esta secuencia retrocede un nivel en el árbol del sistema de archivos.
2. Concatenación de secuencias: Al enviar tres o cuatro repeticiones (`'../../..'`), el puntero de ruta retrocede desde el subdirectorio de almacenamiento (por ejemplo, `'/var/www/images/'`) hacia `'/var/www/'`, luego hacia `'/var/'`, y finalmente hacia la raíz del sistema `'/'`. En entornos UNIX, emitir secuencias de retroceso adicionales una vez alcanzada la raíz no produce error ni afecta el destino, manteniéndose siempre en `'/'`.
3. `'etc/passwd'`: Especifica la ruta relativa que, a partir del directorio raíz ya alcanzado, conduce al archivo estándar del sistema donde se definen las cuentas de usuario, sus identificadores (UID/GID), directorios de inicio (home) y shells por defecto.

Al no existir sanitización ni validación contra listas blancas, la función del backend concatena la entrada de forma cruda (ej. `'/var/www/images/' + '../../etc/passwd'`), resolviendo la ruta canónica en `'/etc/passwd'` y forzando a la API del sistema de archivos a leer y devolver un archivo sensible del sistema operativo.

Mitigación recomendada

Para mitigar eficazmente esta vulnerabilidad se debe implementar una estrategia de defensa en profundidad compuesta por los siguientes controles técnicos:

1. Eliminación del paso directo de entradas a APIs del sistema de archivos: La solución más robusta consiste en evitar que el usuario manipule nombres o rutas de archivos. Se recomienda asociar cada recurso a un identificador único seguro (como un UUID o clave primaria numérica) almacenado en base de datos, resolviendo la ruta del archivo internamente sin intervención del cliente.
2. Validación estricta por lista blanca: En caso de requerir el ingreso de nombres de archivo, validar que el valor contenga exclusivamente caracteres alfanuméricos simples y extensiones autorizadas predeterminadas, rechazando de tajo cualquier carácter separador de ruta ('/', '\') o secuencias de puntos.
3. Canonicalización y verificación de ruta base: Antes de acceder a cualquier recurso, utilizar las funciones nativas del lenguaje o entorno para resolver la ruta absoluta canónica del archivo solicitado (por ejemplo, 'getCanonicalPath()' en Java o 'realpath()' en PHP) y verificar programáticamente que la ruta resultante comience estrictamente con la ruta del directorio base autorizado.
4. Principio de menor privilegio: Asegurar que el proceso del servidor web o demonio de la aplicación se ejecute con una cuenta de servicio de privilegios mínimos (chroot / jail o contenedor aislado), restringiendo a nivel de permisos de sistema operativo la lectura sobre directorios críticos (/etc, /root, /var/log).
5. Implementación de WAF y auditorías continuas: Desplegar un Web Application Firewall con firmas y reglas de inspección profunda capaces de bloquear patrones comunes de evasión de rutas, complementándolo con análisis estático de código (SAST) y pruebas de penetración periódicas.